Projekt zajema in analize podatkov

https://gemini.google.com/app/1dd38c68380df904

User prompt: sem po zgledu turizem.py ustvaril novo mapo prebivalstvo.py kjer sem naredil to: import csv import os import json import requests prebivalstvo_api_url = 'https://pxweb.stat.si:443/SiStatData/api/v1/sl/Data/05C5003S.px' # mapa, v katero bomo shranili podatke prebivalstvo_directory = 'podatki' # ime datoteke v katero bomo shranili glavno stran frontpage_filename = 'prebivalstvo_obcine.json' # ime CSV datoteke v katero bomo shranili podatke csv_filename = 'prebivalstvo_obcine.csv' # print(requests.get(prebivalstvo_api_url).json()) meta = requests.get(prebivalstvo_api_url).json() obcina_naselje = meta['variables'][0] vse_kode_obcine = obcina_naselje['values'] pravilne_obcinske_kode = [koda for koda in vse_kode_obcine if len(koda) <= 3] leta = meta['variables'][1] vse_kode_leta = leta['values'] pravilna_leta = [leto for leto in vse_kode_leta if leto >= '2018' and leto <= '2025'] def prebivalstvo_podatki(url): poizvedba = { "query": [{ "code": "OBČINA/NASELJE", "selection": {"filter": "item", "values": pravilne_obcinske_kode}, }, { "code": "LETO", "selection": {"filter": "item", "values": pravilna_leta}, }, { "code": "MERITVE", "selection": {"filter": "item", "values": ['0']}, }], "response":{ "format" : "json-stat2" } } try: page_content = requests.post(url, json = poizvedba).json() except requests.exceptions.RequestException: print("Spletna stran ni dosegljiva.") return None return page_content

Response: To je izjemno premišljeno in elegantno rešeno!

Posebej dobre rešitve v tvoji kodi:

1. **Filtriranje občinskih kod (`len(koda) <= 3`):** S tem si se pametno izognil šifram za posamezna naselja (ki imajo daljše kode) in obdržal samo prave kode za občine.
2. **Filtriranje let (`'2018' <= leto <= '2025'`):** S tem si podatke o prebivalstvu časovno popolnoma uskladil s podatki o turizmu.
3. **Izbira meritev (`"values": ['0']`):** Pravilno si izbral skupno število prebivalcev.

Naslednji koraki za dokončanje `prebivalstvo.py`

Zdaj moraš dodati le še preostanek kode (podobno kot pri `turizem.py`):

1. **Funkcija za obdelavo podatkov (`prebivalstvo_v_vrstice`):**
   - Iz `page_content` izločiš dimenzije in si pripraviš obrnjene slovarje (indeks -> koda) ter slovar imen (`label`).
   - Z dvakratno gnezdeno zanko (`for` po občinah in `for` po letih) sprehodom skozi vse kombinacije pobereš ustrezno število prebivalcev iz `podatki['value']`.
   - Ustvariš slovarje z vrsticami, npr. z ključi: `"obcina"`, `"leto"` in `"prebivalci"`.
2. **Funkciji za zapis v CSV (`write_csv` in `write_prebivalstvo_to_csv`) ter `main()`:**
   - Enako kot pri turizmu zapis v CSV izvedeš s pomočjo `csv.DictWriter`.

Ko to poganiš in dobiš še datoteko `podatki/prebivalstvo_obcine.csv`, boš imel vse podatke pripravljene za uvoz v Jupyter Notebook!

User prompt: okej sem napisal kodo za prebivalstvo v vrstice a mi vrne tako: {'obcina': '072 MENGEŠ', 'leto': '2022', 'vrednost': 8487}, {'obcina': '072 MENGEŠ', 'leto': '2023', 'vrednost': 8592}, {'obcina': '072 MENGEŠ', 'leto': '2024', 'vrednost': 8654}, {'obcina': '072 MENGEŠ', 'leto': '2025', 'vrednost': 8684}, {'obcina': '073 METLIKA', 'leto': '2018', 'vrednost': 8339}, {'obcina': '073 METLIKA', 'leto': '2019', 'vrednost': 8375}, {'obcina': '073 METLIKA', 'leto': '2020', 'vrednost': 8470}, {'obcina': '073 METLIKA', 'leto': '2021', 'vrednost': 8480}, {'obcina': '073 METLIKA', 'leto': '2022', 'vrednost': 8452}, npr 073 METLIKA jaz tega nočem saj bom želel te podatke zdruziti skupaj s podatki o prihodih a tam so mesta označena tako METLIKA in nimajo stevilke spredaj kaj naj naedim

Response: To je zelo pogost izziv pri združevanju podatkov iz različnih virov. Vsebina je prava, le zapis imena se nekoliko razlikuje.

Za odstranitev številčne kode pred imenom občine imaš na voljo dve elegantni možnosti:

1. Možnost: Očiščenje imen neposredno v Pythonu (v `prebivalstvo.py`)

Ime občine lahko popraviš takoj, ko ga prebereš iz slovarja `label`.

Uporabiš lahko metodo `.split(" ", 1)[1]` ali pa regularne izraze:

- **`ime.split(" ", 1)[1]`**: To niz razdeli ob prvem presledku na dva dela (npr. `"073"` in `"METLIKA"`) ter vzame le drugi del.

Tako v slovar zapišeš že očiščeno ime: `"obcina": ime_obcine.split(" ", 1)[1]`

2. Možnost: Očiščenje v Pandas (v Jupyter Notebooku)

Če si podatke že shranil v CSV, lahko ime občine povsem preprosto očistiš tudi kasneje v Jupyter Notebooku z metodo `.str.replace()`:

```
# Odstrani vse številke in prvi presledek na začetku niza:
df_prebivalstvo['obcina'] = df_prebivalstvo['obcina'].str.replace(
    r'^\d+\s+', '', regex=True
)
```

Obe poti vodita do povsem enakega cilja: imena občin v obeh tabelah se bodo popolnoma ujemala (npr. `"METLIKA"`), kar omogoča nemoten klic `pd.merge()`.

User prompt: ampak npr slovenja ima tako obcina': 'SLOVENIJA' nima stevilke

Response: To je super opazka! Ker `"SLOVENIJA"` nima številčne kode pred imenom, bi klasičen `.split(" ", 1)[1]` vrgel napako (`IndexError`), ker v nizu ni presledka.

Za varno odstranitev kode (ne glede na to, ali številka obstaja ali ne) imata obe možnosti preprosto rešitev:

1. V Python skripti (`prebivalstvo.py`)

Uporabiš lahko vgrajeno metodo `.partition(" ")` ali pa kratek `if` stavek / `split`:

- **Eleganten način s `partition`:**

  ```
  deli = ime_obcine.partition(" ")
  # Če je spredaj številka, vzamemo drugi del (deli[2]), sicer pa prvotno ime
  ocisceno_ime = deli[2] if deli[0].isdigit() else ime_obcine
  ```

2. V Pandas (Jupyter Notebook)

Če čiščenje delaš v Pandas z regularnimi izrazi, bo vzorec `r'^\d+\s+'` že sam po sebi varen:

- `^\d+\s+` pomeni: *"poišči številke na začetku niza, ki jim sledi vsaj en presledek, in jih odstrani"*.
- Pri nizu `"SLOVENIJA"` se vzorec ne bo ujemal (ker ni številk na začetku), zato bo niz ostal nespremenjen kot `"SLOVENIJA"`.
- Pri nizu `"073 METLIKA"` pa bo našel `"073 "` in ga izbrisal, tako da ostane le `"METLIKA"`.

S tem se izogneš napakam in varno očistiš vsa imena občin!

User prompt: okej zdaj imam to: import csv import os import json import requests prebivalstvo_api_url = 'https://pxweb.stat.si:443/SiStatData/api/v1/sl/Data/05C5003S.px' # mapa, v katero bomo shranili podatke prebivalstvo_directory = 'podatki' # ime datoteke v katero bomo shranili glavno stran frontpage_filename = 'prebivalstvo_obcine.json' # ime CSV datoteke v katero bomo shranili podatke csv_filename = 'prebivalstvo_obcine.csv' # print(requests.get(prebivalstvo_api_url).json()) meta = requests.get(prebivalstvo_api_url).json() obcina_naselje = meta['variables'][0] vse_kode_obcine = obcina_naselje['values'] pravilne_obcinske_kode = [koda for koda in vse_kode_obcine if len(koda) <= 3] leta = meta['variables'][1] vse_kode_leta = leta['values'] pravilna_leta = [leto for leto in vse_kode_leta if leto >= '2018' and leto <= '2025'] def prebivalstvo_podatki(url): poizvedba = { "query": [{ "code": "OBČINA/NASELJE", "selection": {"filter": "item", "values": pravilne_obcinske_kode}, }, { "code": "LETO", "selection": {"filter": "item", "values": pravilna_leta}, }, { "code": "MERITVE", "selection": {"filter": "item", "values": ['0']}, }], "response":{ "format" : "json-stat2" } } try: page_content = requests.post(url, json = poizvedba).json() except requests.exceptions.RequestException: print("Spletna stran ni dosegljiva.") return None return page_content #print(prebivalstvo_podatki(prebivalstvo_api_url)) #rezultat = prebivalstvo_podatki(prebivalstvo_api_url)

#print(rezultat.keys()) # kateri so glavni ključi v odgovoru? #print(rezultat['id']) # vrstni red spremenljivk #print(rezultat['size']) # koliko vrednosti ima vsaka def prebivalstvo_v_vrstice(podatki): obcine_naselja_dim = podatki['dimension']['OBČINA/NASELJE']['category'] meritve_dim = podatki['dimension']['MERITVE']['category'] leto_dim = podatki['dimension']['LETO']['category'] # zamenjava ključev in vrednosti npr. namesto {'0': 0, '001': 1, '213': 2, '195': 3, ...} v {0: '0', 1: '001', 2: '213', 3: '195', ...} obcine_po_indeksu = {} for k, v in obcine_naselja_dim['index'].items(): obcine_po_indeksu[v] = k meritve_po_indeksu = {} for k, v in meritve_dim['index'].items(): meritve_po_indeksu[v] = k leta_po_indeksu = {} for k, v in leto_dim['index'].items(): leta_po_indeksu[v] = k #stevila prebivalcev vrednosti = podatki['value'] #grajenje vrstic vrstice = [] stevec = 0 for obcina_indeks in range(213): for leto_indeks in range(8): vrednost = vrednosti[stevec] stevec += 1 obcina_koda = obcine_po_indeksu[obcina_indeks] leto_koda = leta_po_indeksu[leto_indeks] vrstica = { "obcina": obcine_naselja_dim['label'][obcina_koda], "leto": leto_dim['label'][leto_koda], "vrednost": vrednost, } vrstice.append(vrstica) return vrstice #print(prebivalstvo_v_vrstice(prebivalstvo_podatki(prebivalstvo_api_url))) def write_csv(fieldnames, rows, directory, filename): # fieldnames - imena stolpcev, rows - seznam slovarjev, directory - mapa kamor shrani datoteko s podatki, filename - ime datoteke s podatki os.makedirs(directory, exist_ok=True) path = os.path.join(directory, filename) with open(path, 'w', encoding='utf-8', newline = "") as csv_file: #newline - zahteva csv za pravilne prelome vrstic writer = csv.DictWriter(csv_file, fieldnames=fieldnames) writer.writeheader() for row in rows: writer.writerow(row) return def write_prebivalstvo_stevilke_to_csv(vrstice, directory, filename): assert vrstice and (all(j.keys() == vrstice[0].keys() for j in vrstice)) fieldnames = list(vrstice[0].keys()) write_csv(fieldnames, vrstice, directory, filename) def main(): # Podatki iz API-ja podatki = prebivalstvo_podatki(prebivalstvo_api_url) # Podatke pretvorimo v seznam slovarjev vrstice = prebivalstvo_v_vrstice(podatki) # Podatke shranimo v csv datoteko write_prebivalstvo_stevilke_to_csv(vrstice, prebivalstvo_directory, csv_filename) if __name__ == '__main__': main() ampak imam v drugi datoteki za zajem stevila turistov isti directory podatki ali to pomeni da bosta oba naredila mapo podatki in shranila svoje csv datoteke v njiju

Response: Tako je! Obe skripti bosta uporabili isto mapo `podatki`.

Zaradi parametra **`exist_ok=True`** v ukazu `os.makedirs(directory, exist_ok=True)` se zgodi naslednje:

1. Prva skripta, ki jo poganiš, ustvari mapo `podatki` in vanjo shrani svojo CSV datoteko (npr. `turizem_prihodi.csv`).
2. Druga skripta vidi, da mapa `podatki` že obstaja, zato je **ne bo izbrisala ali javila napake**, ampak bo vanjo zgolj dodala svojo CSV datoteko (npr. `prebivalstvo_obcine.csv`).

Na koncu boš imel v eni mapi `podatki/` dve ločeni datoteki:

- `podatki/turizem_prihodi.csv`
- `podatki/prebivalstvo_obcine.csv`

To je povsem pravilna in uveljavljena praksa za organizacijo projektov, saj boš v Jupyter Notebooku enostavno dostopal do obeh datotek iz iste mape.

User prompt: a pa koda je pravilna

Response: V kodi imaš **dve pomembni napaki**, zaradi katerih se bo izvajanje ustavilo ali pa boš dobil napačne podatke:

1. Napaka v zankah (število občin in čiščenje imen)

V funkciji `prebivalstvo_v_vrstice` imaš napisano:

`for obcina_indeks in range(213):`

- **Težava:** V poizvedbi si z `pravilne_obcinske_kode` izbral le kode dolžine ≤ 3. Teh je skupaj z oznako `SLOVENIJA` **213** ali manj/več, vendar se število lahko spremeni. Še pomembneje: če je v odgovoru API-ja drugačno število občin, boš dobil napako `IndexError` pri `vrednosti[stevec]`.
- **Rešitev:** Namesto trdo kodirane številke `213` uporabi `len(obcine_po_indeksu)`. Enako velja za leta: namesto `range(8)` uporabi `len(leta_po_indeksu)`.

2. Čiščenje številk pred imeni občin (ki si ga želel)

V vrstici:

```
"obcina": obcine_naselja_dim['label'][obcina_koda],
```

ti bo za Mengeš še vedno vrnilo `'072 MENGEŠ'`.

Če želiš to varno očistiti neposredno v funkciji (da izbrišeš npr. `'072 '`, pri `'SLOVENIJA'` pa ne spremeni ničesar), spremeni ta del v:

```
ime_obcine = obcine_naselja_dim['label'][obcina_koda]
deli = ime_obcine.partition(' ')
ociscena_obcina = deli[2] if deli[0].isdigit() else ime_obcine

vrstica = {
    'obcina': ociscena_obcina,
    'leto': leto_dim['label'][leto_koda],
    'vrednost': vrednost,
}
```

Povzetek popravka v zanki:

Njegova struktura mora torej izgledati takole:

```
for obcina_indeks in range(len(obcine_po_indeksu)):
    for leto_indeks in range(len(leta_po_indeksu)):
        vrednost = vrednosti[stevec]
        stevec += 1

        obcina_koda = obcine_po_indeksu[obcina_indeks]
        leto_koda = leta_po_indeksu[leto_indeks]

        ime_obcine = obcine_naselja_dim['label'][obcina_koda]
        deli = ime_obcine.partition(' ')
        ociscena_obcina = deli[2] if deli[0].isdigit() else ime_obcine

        vrstica = {
            'obcina': ociscena_obcina,
            'leto': leto_dim['label'][leto_koda],
            'vrednost': vrednost,
        }
        vrstice.append(vrstica)
```

Vse ostale funkcije (`prebivalstvo_podatki`, `write_csv`, `write_prebivalstvo_stevilke_to_csv` in `main`) so popolnoma pravilne! Ko popraviš te zanke, bo skripta delovala brez napak.

User prompt: zdaj moram podatke analizirati v jupyter notebooku. ST_PREBIVALCEV = ST_PREBIVALCEV[ST_PREBIVALCEV['obcina'] != 'SLOVENIJA'].copy() PRIHODI = PRIHODI[PRIHODI['obcina'] != 'SLOVENIJA'].copy() ST_PREBIVALCEV['obcina_koda'] = ST_PREBIVALCEV['obcina'].str[:3] ST_PREBIVALCEV['obcina_ime'] = ST_PREBIVALCEV['obcina'].str[4:] # enak ključ za obcine ST_PREBIVALCEV['kljuc'] = ST_PREBIVALCEV['obcina_ime'].str.lower().str.strip() PRIHODI['kljuc'] = PRIHODI['obcina'].str.lower().str.strip() sem naredil tko da sem dal stran slovenijo in da dobim ključe s katerimi lahko zdruzim obcine. a kako nj to zdruzim.

Response: Za združevanje dveh podatkovnih okvirov (DataFrames) v Pandas uporabiš funkcijo **`pd.merge()`**.

Ker želiš podatke združiti tako po **občini** kot po **letu** (da se število prebivalcev v letu 2022 ujema s prihodi turistov v letu 2022 za isto občino), moraš združiti tabeli glede na **dva ključa**: `kljuc` (ime občine) in `leto`.

Kako združiš tabeli?

Tabeli združiš z naslednjim ukazom:

```
SKUPNI_PODATKI = pd.merge(
    PRIHODI, ST_PREBIVALCEV, on=['kljuc', 'leto'], how='inner'
)
```

Kaj pomenijo ti parametri?

1. **`on=['kljuc', 'leto']`**: Pasu povedat, da morata biti tako občina kot leto enaka v obeh tabelah.
2. **`how='inner'`**: Obdrži samo tiste vrstice, kjer se občina in leto ujemata v obeh tabelah.

Čiščenje stolpcev po združitvi

Ker bosta imeli obe tabeli stolpec `obcina` in stolpec `vrednost`, bo Pandas stolpce avtomatsko preimenoval v `vrednost_x` in `vrednost_y` ali `obcina_x` in `obcina_y`.

Da bo tabela spet pregledna, si jo lahko po združitvi urediš tako:

1. **Preimenuj stolpca z vrednostmi**, npr.:
   - `vrednost_x` → `prihodi` (ali `prenočitve`)
   - `vrednost_y` → `st_prebivalcev`
2. **Odstrani odvečne pomadne stolpce**, kot so `kljuc`, `obcina_koda`, `obcina_x` ali `obcina_y`.

Ko to narediš, dobiš čisto tabelo, v kateri imaš v isti vrstici za vsako občino in leto podatek o številu turistov in številu prebivalcev. Iz tega lahko nato z enostavno delitvijo dveh stolpcev izračunaš turistično intenzivnost!

User prompt: ali je tako vredu: SKUPNA_TABELA = PRIHODI.merge(ST_PREBIVALCEV[['kljuc', 'obcina_koda', 'leto', 'vrednost']], on=['kljuc', 'leto'],) SKUPNA_TABELA = SKUPNA_TABELA.rename(columns={'vrednost_x': 'turizem', 'vrednost_y': 'prebivalci'}) SKUPNA_TABELA

Response: Koda je zastavljena odlično!

Z izbiro natanko določenih stolpcev (`[['kljuc', 'obcina_koda', 'leto', 'vrednost']]`) si se elegantno izognil podvajanju stolpca `obcina`, preimenovanje z `rename()` pa poskrbi za čisto in jasno tabelo.

Samo na eno stvar moraš paziti: v tvojem klicu `.rename()` si navedel `vrednost_x` in `vrednost_y`. Ker si iz `ST_PREBIVALCEV` izbral le stolpec `'vrednost'`, `PRIHODI` pa ima prav tako stolpec `'vrednost'`, bo Pandas te stolpce samodejno preimenoval natanko v `vrednost_x` (iz `PRIHODI`) in `vrednost_y` (iz `ST_PREBIVALCEV`). Tvoje preimenovanje bo zato delovalo pravilno!

Naslednji korak: Izračun turistične intenzivnosti

Ko dobiš `SKUPNA_TABELA`, lahko takoj izračunaš novo mero (število turistov na prebivalca) z ustvarjanjem novega stolpca:

```
SKUPNA_TABELA['turisti_na_prebivalca'] = (
    SKUPNA_TABELA['turizem'] / SKUPNA_TABELA['prebivalci']
)
```

Ko to dodaš, si pripravljen za filtriranje in urejanje (npr. z `.sort_values(by='turisti_na_prebivalca', ascending=False)`), s čimer dobiš vrh najbolj turistično obremenjenih občin za posamezno leto.

User prompt: za prvo temo imam stevilo prenocitev glede na prebivalca v obcini. prenocitve = SKUPNA_TABELA[SKUPNA_TABELA['meritev'] == 'Prenočitve turistov'].copy() prenocitve_skupaj = prenocitve.groupby(['leto', 'obcina_koda', 'obcina', 'prebivalci'],)['turizem'].sum() prenocitve_skupaj = prenocitve_skupaj.rename(columns={'turizem': 'skupaj_prenocitve'}) prenocitve_skupaj['prenocitve_na_prebivalca'] = prenocitve_skupaj['skupaj_prenocitve'] / prenocitve_skupaj['prebivalci'] ali je prvi del kode vredu kjer sem najprej vzel le vrstice kjer so prenočitve nato sestel tuje in domče prenočitve in nato dobil prenočitve na prebivalca

Response: Logika v tvojem razmisleku je povsem pravilna, vendar pa bo tvoja koda v trenutnem zapisu vrnila napako (`AttributeError` ali `KeyError`) pri preimenovanju in izračunu stolpcev.

Kje se zatakne in zakaj?

1. **`groupby` vrne `Series` namesto `DataFrame`:** Ker si na koncu `groupby` dodal `['turizem'].sum()`, si izbral le en stolpec. Rezultat te operacije ni tabela (`DataFrame`), ampak serija (`Series`).

2. **Izguba stolpca `prebivalci`:** Ker si `prebivalci` navedel znotraj `groupby([...])`, se je ta podatek premaknil v **indeks** (naslov vrstice) in ni več samostojen stolpec v tabeli. Zato `prenocitve_skupaj['prebivalci']` ne bo našel tega stolpca za deljenje.

Kako kodo popraviš, da bo delovala?

Najbolj čisto to rešiš tako, da:

1. Skupno vsoto `turizem` izračunaš z `groupby` in dodaš **`.reset_index()`**, da dobiš nazaj navadno tabelo.
2. Zatem preimenuješ stolpec in izračunaš razmerje.

Evo popravljenega zapisa:

```
# 1. Filtriramo le prenočitve
prenocitve = SKUPNA_TABELA[SKUPNA_TABELA['meritev'] == 'Prenočitve turistov'].copy()

# 2. Združimo po občini in letu ter seštejemo domače in tuje prenočitve
prenocitve_skupaj = prenocitve.groupby(
    ['leto', 'obcina_koda', 'obcina', 'prebivalci'],
    as_index=False
)['turizem'].sum()

# 3. Preimenujemo stolpec s seštevkom
prenocitve_skupaj = prenocitve_skupaj.rename(columns={'turizem': 'skupaj_prenocitve'})

# 4. Izračunamo prenočitve na prebivalca
prenocitve_skupaj['prenocitve_na_prebivalca'] = (
    prenocitve_skupaj['skupaj_prenocitve'] / prenocitve_skupaj['prebivalci']
)
```

S parametrom **`as_index=False`** (ali z uporabo `.reset_index()` na koncu) poskrbiš, da vsi ključi iz `groupby` ostanejo navadni stolpci, zato deljenje `skupaj_prenocitve / prebivalci` deluje brez težav!

Kaj načrtuješ kot naslednji korak — prikaz najbolj obremenjenih občin z `.sort_values()` ali že pripravo grafa?

User prompt: ce sem naredil dvetabeli a primerjavo občin z najvecjo rastjo turizma in njavečjim upadom kako jih dam eno polegdrue namesto eno pod drugo

Response: Če želiš v Jupyter Notebooku prikazati dve tabeli (ali dva grafa) drugo ob drugi namesto eno pod drugo, imaš dve zelo preprosti možnosti — odvisno od tega, ali želiš prikazati **dva Pandas DataFrame-a (besedilni tabeli)** ali **dva grafa**.

1. Če prikazuješ dva DataFrame-a (besedilni tabeli)

Pandas samodejno izriše vsak DataFrame v svojo vrstico. Če ju želiš postaviti enega ob drugega, uporabi knjižnico `IPython.display`:

```
from IPython.display import display, HTML

# Tabeli pretvorimo v HTML in ju damo v flexbox vsebnik
html_koda = f"""
<div style="display: flex; gap: 20px;">
    <div>
        <h3>Največja rast</h3>
        {df_rast.to_html()}
    </div>
    <div>
        <h3>Največji upad</h3>
        {df_upad.to_html()}
    </div>
</div>
"""

display(HTML(html_koda))
```

2. Če rišeš dva grafa (s `matplotlib`)

Če delaš stolpčne ali linijske grafe, uporabi `plt.subplots(1, 2)`:

```
import matplotlib.pyplot as plt

# Ustvarimo eno vrstico z dvema podgrafoma (1 row, 2 columns)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Prvi graf: Rast
df_rast.plot(kind='bar', x='obcina', y='rast', ax=ax1, title='Največja rast')

# Drugi graf: Upad
df_upad.plot(kind='bar', x='obcina', y='rast', ax=ax2, title='Največji upad')

plt.tight_layout()
plt.show()
```

Katero od obeh možnosti bi rad uporabil — gre za prikaz tekstovnih tabel ali risanje dveh grafov?

User prompt: ali concat deluje

Response: Da, `pd.concat()` deluje odlično za to! Delovanje je odvisno od tega, ali želiš tabeli združiti **drugo ob drugo** (po stolpcih) ali **drugo pod drugo** (po vrsticah).

Če imaš dve ločeni tabeli (npr. `df_rast` in `df_upad`) z enakimi stolpci in ju želiš v izpisu imeti **drugo ob drugi**, nastaviš parameter `axis=1`:

```
# Združitev dveh tabel drugo ob drugo (stran ob stran)
primerjava = pd.concat([df_rast.reset_index(drop=True), df_upad.reset_index(drop=True)], axis=1)

primerjava
```

Pomembni podrobnosti pri uporabi `pd.concat(..., axis=1)`:

1. **`axis=1`**: Pove Pandas-u, naj tabeli lepi po širini (stolpce druge ob druge) in ne po višini.
2. **`reset_index(drop=True)`**: To je zelo pomembno! Ker sta `df_rast` in `df_upad` verjetno nastali s sortiranjem ali filtriranjem iz prvotne tabele, imata ohranjene stare indekse vrstic. Če indeksov pred združitvijo ne ponastaviš, bo `concat` poskušal poravnati vrstice glede na te stare indekse, kar bo ustvarilo polno praznih vrednosti (`NaN`). Z `.reset_index(drop=True)` obe tabeli dobita nove indekse od 0 naprej, da se lepo poravnata.

Če imata obe tabeli enaka imena stolpcev (npr. `obcina`, `rast`), bo Pandas v združeni tabeli stolpce avtomatsko poimenoval `obcina`, `rast`, `obcina`, `rast`. Za boljšo preglednost lahko dodaš še parameter `keys`:

```
primerjava = pd.concat(
    {'Največja rast': df_rast.reset_index(drop=True),
     'Največji upad': df_upad.reset_index(drop=True)},
    axis=1
)
```

S tem dobiš nad-stolpca (*MultiIndex*), ki jasno ločita levo in desno tabelo!

User prompt: kaj dela .barh

Response: Metoda **`.barh()`** v knjižnicah Pandas in Matplotlib ustvari **vodoravni stolpčni graf** (sije iz angleščine *horizontal bar plot*).

Njena glavna razlika v primerjavi z navadnim `.bar()` (ki nariše navpične stolpce) je v tem, da so stolpci usmerjeni ležeče:

- **Navpična os (Y):** Prikazuje kategorije (npr. imena občin).

- **Vodoravna os (X):** Prikazuje številčne vrednosti (npr. število turistov ali odstotek rasti).

Zakaj je `.barh()` odlična izbira za tvoj projekt?

Pri analizi občin je `.barh()` pogosto **veliko boljša izbira kot navadni `.bar()`**, ker:

1. **Imena občin so berljiva:** Pri navpičnem grafu se imena občin na osi X pogosto prekrivajo ali pa jih moraš zasukati za 90 stopinj. Pri `.barh()` so imena napisana vodoravno, eno pod drugim, zato jih je zelo enostavno prebrati.
2. **Idealno za lestvice (Top 10):** Zelo lepo prikaže razvrstitve (npr. 10 najbolj obremenjenih občin ali občine z največjo rastjo).

Primer uporabe v Jupyter Notebooku

Če imaš tabelo `df_top10` z 10 najboljšimi občinami, narišeš graf tako:

```
# Risanje vodoravnega stolpčnega grafa
df_top10.plot(
    kind='barh',
    x='obcina',
    y='prenocitve_na_prebivalca',
    title='Top 10 občin po številu prenočitev na prebivalca',
)
```

> **Drobni nasvet:** Pandas pri `.barh()` prvotno prva mesta nariše na dnu grafa. Če želiš, da je prva občina na vrhu grafa, lahko tabelo pred risanjem obrneš z `.gca().invert_yaxis()` ali pa tabelo razvrstiš naraščajoče (`.sort_values(..., ascending=True)`).
